

Typer CLI for Database Management

<https://gemini.google.com/app/a316238f004aa19a>

User prompt: # main entry point # from config import settings import pyodbc import typer from utils import db_helper import process app = typer.Typer(help="Утилита управления движком (Engine CLI)", add_completion=False) @app.command() def init(): """ system init """ db_helper.create_table('_Reference133') @app.command() def upd(): """ system update """ typer.echo('upd done') if __name__ == "__main__": app()

Response: It looks like you shared a Python script using the typer library to build a command-line interface (CLI) for your application, along with some database helper functions.

How can I help you with this code today? For example, I can help you:

- **Add new commands** or arguments to your Typer app.
- **Refactor or improve** error handling and database connection management.
- **Implement the logic** for your upd() or process modules.
- **Add logging and configuration** using settings (which is currently commented out).

Let me know what you'd like to do next!

User prompt: # main entry point # from config import settings import pyodbc import typer from utils import db_helper import process app = typer.Typer(help="Утилита управления движком (Engine CLI)", add_completion=False) @app.command() def init(): """ system init """ db_helper.create_table('_Reference133') @app.command() def upd(): """ system update """ typer.echo('upd done') if __name__ == "__main__": app() как сюда добавить логи в файл и в бд и алерт на почту, если писание логов упало или сервис упал (логи живы) но крэш был реально критический

Response: Для решения этой задачи лучше всего использовать встроенный модуль logging для записи логов в файл, написать небольшой вспомогательный метод для записи логов в базу данных через ваш db_helper, а также добавить блок try/except с отправкой уведомлений на почту (через стандартный smtplib) при критических сбоях.

Ниже представлен обновленный и структурированный вариант вашего скрипта с добавлением логирования и алертинга.

Обновленный код (main.py)

```
# main entry point
# from config import settings
import logging
import smtplib
from email.message import EmailMessage
import traceback
import pyodbc
import typer
from utils import db_helper
import process

# --- НАСТРОЙКА ЛОГИРОВАНИЯ ---
logging.basicConfig(
    filename="app.log",
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    encoding="utf-8"
)
logger = logging.getLogger("EngineCLI")

app = typer.Typer(
    help="Утилита управления движком (Engine CLI)", add_completion=False
)

# --- ВСПОМОГАЩИЕ ФУНКЦИИ ---

def log_to_db(level: str, message: str):
    """Безопасная запись лога в базу данных."""
    try:
        # Предполагаем, что у вас в db_helper есть метод для вставки логов,
        # например: db_helper.insert_log(level, message)
        db_helper.insert_log(level, message)
    except Exception as e:
        # Если запись в БД упала, пишем об этом в файловый лог и кидаем алерт
        err_msg = f"КРИТИЧЕСКИЙ СБОЙ: Не удалось записать лог в БД! Ошибка: {e}"
        logger.critical(err_msg)
        send_alert("Сбой системы логирования БД", err_msg)

def send_alert(subject: str, body: str):
    """Отправка email-алерта администратору при критическом сбое."""
```

```

# Настройки почты (лучше вынести в config/settings)
SMTP_SERVER = "smtp.example.com"
SMTP_PORT = 587
SMTP_USER = "user@example.com"
SMTP_PASSWORD = "your_password"
ADMIN_EMAIL = "admin@example.com"

try:
    msg = EmailMessage()
    msg.set_content(body)
    msg["Subject"] = f"[ALERT] {subject}"
    msg["From"] = SMTP_USER
    msg["To"] = ADMIN_EMAIL

    # Отправка (замените на SSL, если требуется порт 465)
    with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
        server.starttls()
        server.login(SMTP_USER, SMTP_PASSWORD)
        server.send_message(msg)

    logger.info("Email-алерт успешно отправлен администратору.")
except Exception as e:
    # Если упала даже отправка почты, фиксируем это хотя бы в файле
    logger.critical( не удалось отправить email-алерт: {e})

# --- КОМАНДЫ CLI ---

@app.command()
def init():
    """
    ...
    system init
    ...
    logger.info("Запуск команды: init")
    log_to_db("INFO", "Запуск инициализации системы (_Reference133)")

    try:
        db_helper.create_table('_Reference133')
        logger.info("Таблица _Reference133 успешно создана/проверена.")
        log_to_db("INFO", "Инициализация успешно завершена.")
    except Exception as e:
        tb = traceback.format_exc()
        err_msg = f"Ошибка при инициализации таблицы: {e}\n{tb}"

        logger.error(err_msg)
        log_to_db("ERROR", str(e))

        # Отправляем алерт при падении важного процесса
        send_alert("Критический сбой в команде init", err_msg)
        raise typer.Exit(code=1)

@app.command()
def upd():
    """
    ...
    system update
    ...
    logger.info("Запуск команды: upd")
    typer.echo('upd done')

if __name__ == "__main__":
    try:
        app()
    except Exception as critical_error:
        # Перехват вообще любого необработанного падения скрипта (краш сервиса)
        tb = traceback.format_exc()
        crash_msg = f"КРИТИЧЕСКИЙ КРАШ СЕРВИСА:\n{tb}"

        logger.critical(crash_msg)
        send_alert("Критический краш Engine CLI", crash_msg)
        raise critical_error

```

Что было реализовано:

- **Логи в файл:** Используется стандартный модуль logging, который пишет все события в файл app.log с указанием даты, времени и уровня важности (INFO, ERROR, CRITICAL).
- **Логи в БД:** Добавлена функция log_to_db, которая обращается к вашему db_helper.

- **Защита системы логирования:** Если функция записи в БД падает (except Exception), код автоматически перехватывает ошибку, записывает её в файловый лог и отправляет почтовый алерт о том, что база данных логов недоступна.
- **Алерты на почту (smtplib):** Функция send_alert отправляет email-уведомление администратору. Она задействована как при сбое записи в БД, так и в блоке try/except на уровне вызова приложения (if __name__ == "__main__"), если произойдет неожиданный критический краш всего скрипта.

Какую почтовую службу (SMTP) планируете использовать для отправки алертов (например, корпоративный сервер или внешние сервисы вроде Yandex/Gmail)?

User prompt: если падает и файловый лог - вообще без валидов для алерта?

Response: Если падает и файловый лог (например, закончилось место на диске, пропали права на запись в директорию, либо «умер» сам диск), то стандартные механизмы записи на диск бессильны. В таком случае приложение физически не сможет создать строку в файле.

Но это **не значит, что вариант с алертом полностью исключен**. Выход есть, и он зависит от уровня надежности, который вам нужен:

1. Перехват в памяти и попытка экстренного вылета наружу

Если сбой произошел в момент записи в файл, можно поймать это исключение и попытаться отправить почту **напрямую**, минуя файловое логирование:

```
try:
    logger.critical("Критическая ошибка системы")
except Exception as file_log_error:
    # Файловый лог умер! Пытаемся экстренно спасти ситуацию:
    emergency_msg = f"ФАТАЛ: Упал даже файловый лог! Ошибка: {file_log_error}"

    try:
        # Сразу шлем почту минуя логи
        send_alert("КРИТИЧЕСКИЙ СБОЙ: Отказ файлового логирования", emergency_msg)
    except Exception as mail_error:
        # Если почта тоже отвалилась — последний рубеж (STDERR)
        import sys
        print(f"ULTIMATE CRASH: {emergency_msg} | Mail error: {mail_error}", file=sys.stderr)
```

2. Системный мониторинг (Watchdog / الخارج)

В продакшене полагаться только на код приложения при падении диска или ОС — рискованно. Если приложение *вообще* зависло, упало по segmentation fault или умерло так, что даже блок if __name__ == "__main__": не успел отработать (например, OOM Killer прикончил процесс из-за нехватки памяти), никакой внутренний скрипт почту не отправит.

Для таких случаев используют **системные внешние страховки**:

- **Systemd (если Linux):** Можно настроить параметр OnFailure= в файле службы (.service). Если сервис падает с ошибкой или с сигналом краша, systemd может запустить отдельный скрипт оповещения (например, отправить письмо или сообщение в Telegram-бот).
- **Мониторинг процессов (Zabbix, Prometheus/Alertmanager, Monit):** Утилите Monit или демону мониторинга можно дать задачу: пинговать ваш сервис или проверять, что процесс жив и пишет в файл последние N минут. Если файл замерл или процесс пропал — внешний агент шлет алерт.

Резюме

Если «умирает» файловый лог, единственное, что может спасти отправку почты изнутри скрипта — это **блок try/except вокруг самой записи в лог**, который мгновенно переключается на отправку тревожного email. А для полной гарантии от полного краха среды лучше дублировать контроль внешними средствами ОС.

User prompt: процесс запускается в windows scheduler

Response: Разпуск процесса через **Windows Task Scheduler (Планировщик заданий)** дает отличный инструмент контроля: планировщик всегда знает, с каким кодом завершился скрипт (успех/ошибка).

Если файловый лог отвалился (диск переполнился, слетели права), полагаться на внутренние файлы бессмысленно. В этом случае связка **Python + Task Scheduler** должна работать по следующей схеме:

1. Внутренняя страховка в Python (если упал даже файловый лог)

Если скрипт падает (даже если лог на диск записать не удалось), код должен попытаться отправить email напрямую из оперативной памяти, вывести ошибку в stderr и завершиться с ненулевым кодом (sys.exit(1)).

Вот как можно оформить главный блок вашего скрипта (main.py):

```
import sys
import traceback

def send_emergency_alert(error_text: str):
```

```

"""Экстренная отправка почты напрямую, минуя файловые логи."""
try:
    # Ваш код отправки через smtplib (без вызова logger)
    pass
except Exception as mail_err:
    print(f"FATAL: Не удалось отправить экстренный email: {mail_err}", file=sys.stderr)

if __name__ == "__main__":
    try:
        app()
    except Exception as critical_error:
        tb = traceback.format_exc()
        crash_msg = f"КРИТИЧЕСКИЙ КРАШ в Windows Task Scheduler:\n{tb}"

        # 1. Пытаемся записать в файл (если файл жив)
        try:
            logger.critical(crash_msg)
        except Exception:
            pass # Файловый лог мертв — игнорируем и идем дальше

        # 2. Шлем почту напрямую
        send_emergency_alert(crash_msg)

        # 3. Возвращаем кодом ошибки для Windows
        sys.exit(1)

```

2. Что делает Windows Task Scheduler?

Планировщик задач в Windows фиксирует код возврата (Exit Code).

- Если скрип завершился штатно — код 0.
- Если сработал `sys.exit(1)` или аварийный краш — код отличный от нуля (например, 1 или -1073741819 для Access Violation).

Как настроить реакцию на сбой снаружи:

1. Через Журнал событий (Event Viewer):

- Планировщик заданий пишет в системный журнал все запуски и падения задач (источник: TaskScheduler, уровни: Ошибка / Предупреждение).
- Вы можете создать триггер на системное событие Windows (Event ID, когда задача завершилась с ошибкой), который запустит другой легкий скрипт (например, PowerShell-скрипт отправки письма).

2. Сторожевой wrapper-скрипт (самый надежный метод для Windows):

Вместо того чтобы запускать ваш `python.exe main.py` напрямую из планировщика, запускайте промежуточный `.bat` или `.ps1` файл (wrapper), который контролирует результат выполнения:

Пример (run.bat):

```

@echo off
C:\path\to\python.exe C:\path\to\main.py %*

if %ERRORLEVEL% NEQ 0 (
    echo Задача завершилась с ошибкой! Код: %ERRORLEVEL%
    :: Здесь можно вызвать PowerShell-команду для отправки письма,
    :: если сам Python даже почту отправить не смог
    powershell -Command "Send-MailMessage -To 'admin@example.com' -From 'bot@example.com' -Subject 'CRITICAL: Er
)

```

Резюме по архитектуре алертов:

- Первый рубеж:** Блок `try/except` внутри Python пытается отправить почту и записать в файл.
- Второй рубеж (если диск с логами умер):** Python ловит сбой файлового лога, игнорирует его, шлет почту напрямую из памяти и делает `sys.exit(1)`.
- Третий рубеж (если Python вообще не смог стартовать или упал намертво):** `.bat / .ps1` файл-обертка в Windows Scheduler перехватывает ненулевой `ERRORLEVEL` и отправляет резервное сообщение через PowerShell.

User prompt: шлет почту напрямую из памяти и делаем sys.exit(1)

Response: Вот готовая реализация этой схемы. Весь код сосредоточен в защитном блоке `if __name__ == "__main__":` и вспомогательной функции экстренной отправки, которая не зависит от файлов и базы данных.

Реализация аварийного перехвата

```

import sys
import traceback

```

```

import smtplib
from email.message import EmailMessage

def send_emergency_alert(error_text: str):
    """
    Прямая отправка письма из оперативной памяти.
    Полностью автономна: не использует файловый лог и БД.
    """
    SMTP_SERVER = "smtp.example.com"
    SMTP_PORT = 587
    SMTP_USER = "user@example.com"
    SMTP_PASSWORD = "your_password"
    ADMIN_EMAIL = "admin@example.com"

    try:
        msg = EmailMessage()
        msg.set_content(error_text)
        msg["Subject"] = "[FATAL CRASH] Engine CLI: критический отказ"
        msg["From"] = SMTP_USER
        msg["To"] = ADMIN_EMAIL

        # Таймаут обязателен, чтобы скрипт не завис навсегда, если SMTP-сервер лег
        with smtplib.SMTP(SMTP_SERVER, SMTP_PORT, timeout=10) as server:
            server.starttls()
            server.login(SMTP_USER, SMTP_PASSWORD)
            server.send_message(msg)

    except Exception as mail_err:
        # Абсолютный тупик: и лог упал, и почта не ушла. Пишем в stderr для Task Scheduler.
        print(f"FATAL: Не удалось отправить аварийный email: {mail_err}", file=sys.stderr)

if __name__ == "__main__":
    try:
        app()
    except Exception as critical_error:
        tb = traceback.format_exc()
        crash_msg = f"КРИТИЧЕСКИЙ КРАШ СЕРВИСА:\n{tb}"

        # Рубеж 1: Пытаемся зафиксировать в штатный файловый лог
        try:
            logger.critical(crash_msg)
        except Exception:
            pass # Файловый лог мертв (нет места/прав) — просто игнорируем

        # Рубеж 2: Шлем почту напрямую из памяти минуя все преграды
        send_emergency_alert(crash_msg)

        # Рубеж 3: Завершаем процесс с кодом ошибки для Windows Task Scheduler
        sys.exit(1)

```

Как это работает на практике:

1. Если падает обычная команда — срабатывает локальный try/except внутри функции (например, в init()).
2. Если происходит необработанный фатальный сбой, управление падает в глобальный except Exception as critical_error.
3. Скрипт пытается сделать logger.critical(). Если **файловый лог тоже умер** (ошибка записи проглатывается через pass), выполнение не прерывается.
4. Вызывается send_emergency_alert(), которая берет текст ошибки из переменной в памяти (tb), собирает пакет и шлет письмо по SMTP.
5. Вызывается sys.exit(1). Планировщик задач Windows фиксирует, что задача завершилась с ненулевым кодом, и это можно дополнительно отловить на уровне ОС (например, через .bat обертку, если потребуется двойная страховка).